Relatório Trabalho - Sistema Centro Esportivo

Integrantes: Camila R. Ribeiro, Helena Vieira e Maria Clara Pastor

Repositório dos códigos: https://github.com/camilaribeirox/centro-esportivo

1. Diagrama Entidade-Relacionamento Conceitual

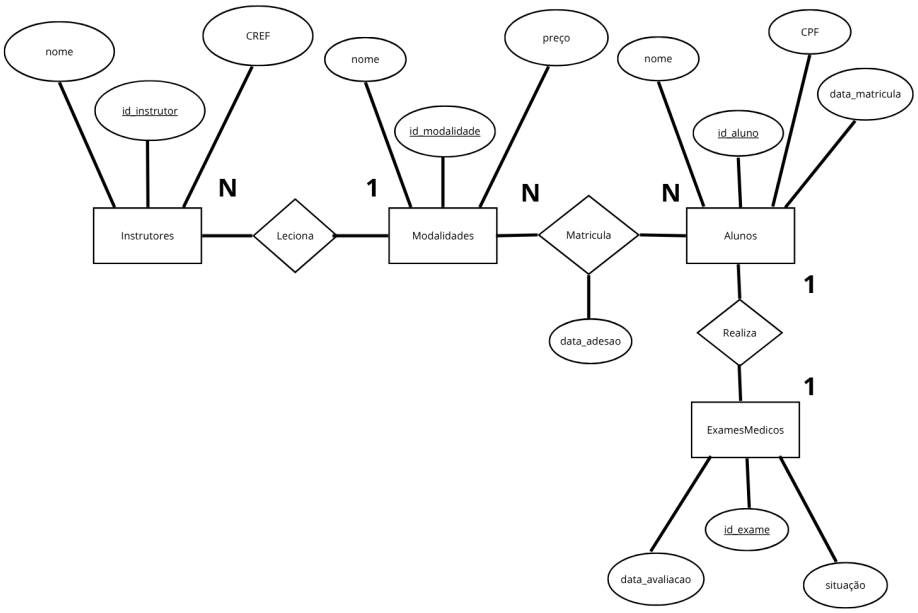


Figura 1: DER conceitual do centro esportivo

É a representação gráfica que descreve o quê o sistema precisa guardar da realidade, sem se preocupar com comandos técnicos, SQL ou qual SGBD será usado. É o diagrama feito para o cliente e analista validarem as regras de negócio.

Entidades: Representadas por retângulos (Instrutores, Modalidades, Alunos, ExamesMedicos).

Relacionamentos: Representados por losangos (Leciona, Matricula, Realiza).

Atributos: Representados por elipses ligadas às entidades, com chaves identificadoras devidamente sublinhadas (id_instrutor, id_modalidade, id_aluno, id_exame).

Atributo de Relacionamento: O atributo data_adesao está associado diretamente ao losango Matricula (relacionamento N:N com atributos próprios).

Posicionamento de Cardinalidades: "A cardinalidade de uma entidade é sempre colocada na linha após o losango":

- Instrutores (N) -- [Leciona] -- (1) Modalidades: 1 instrutor leciona 1 modalidade; 1 modalidade pode reunir N instrutores.

- Modalidades (N) -- [Matricula] -- (N) Alunos: 1 modalidade reúne N alunos; 1 aluno pode cursar N modalidades.
- Alunos (1) -- [Realiza] -- (1) ExamesMedicos: 1 aluno realiza 1 exame médico cadastrado; 1 exame pertence a 1 aluno.

2. Modelo Lógico Relacional

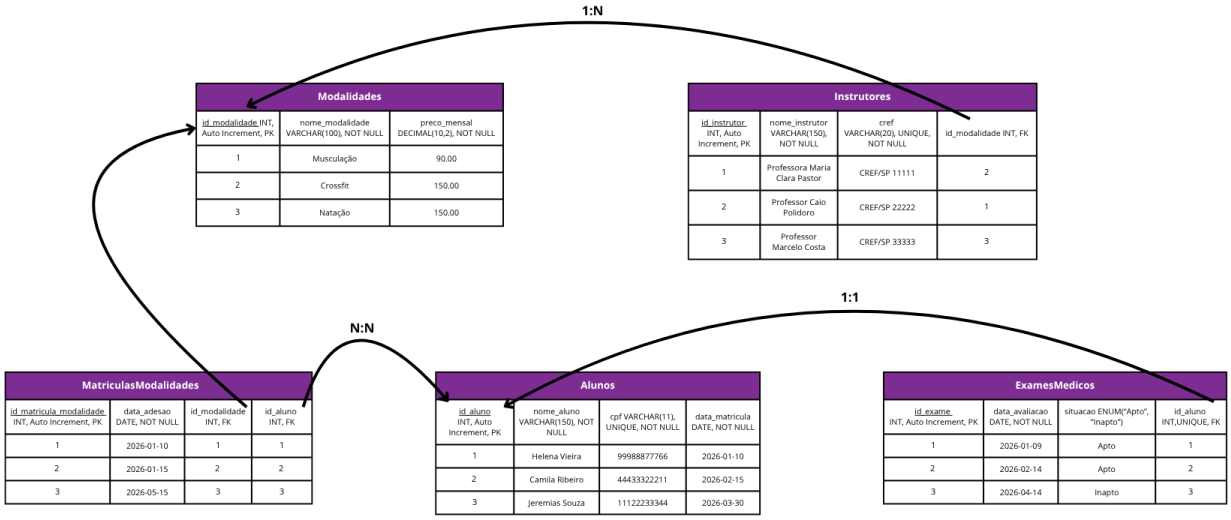


Figura 2: modelo lógico relacional com dados de teste

É a representação técnica direta de como os dados serão organizados no SGBD Relacional (MySQL). Ele traduz as entidades do DER em tabelas bidimensionais com colunas e linhas.

Tabelas e Tipos: Todas as 5 tabelas possuem suas colunas tipadas (INT, VARCHAR, DECIMAL, DATE, ENUM), com PKs com Auto Increment e chaves identificadas.

Dados de Teste: Tuplas reais de teste (ex: Professora Maria Clara Pastor, Helena Vieira, Jeremias Souza, etc.).

Relação 1:N (Modalidades e Instrutores): A tabela do lado N (Instrutores) contém a Chave Estrangeira id_modalidade INT, FK apontando para Modalidades.

Relação N:N (Alunos e Modalidades): Tabela associativa MatriculasModalidades contendo as Chaves Estrangeiras id_modalidade e id_aluno e o atributo da adesão data_adesao DATE.

Relação 1:1 (Alunos e ExamesMedicos): Adicionamos id_aluno INT, UNIQUE, FK na tabela ExamesMedicos. A restrição UNIQUE é o requisito essencial para garantir que a relação seja estritamente 1 para 1.

Setas de Integridade Referencial: Indicam perfeitamente a chave de origem apontando para a chave primária de destino.

3. Diagrama de Classes de Entidades

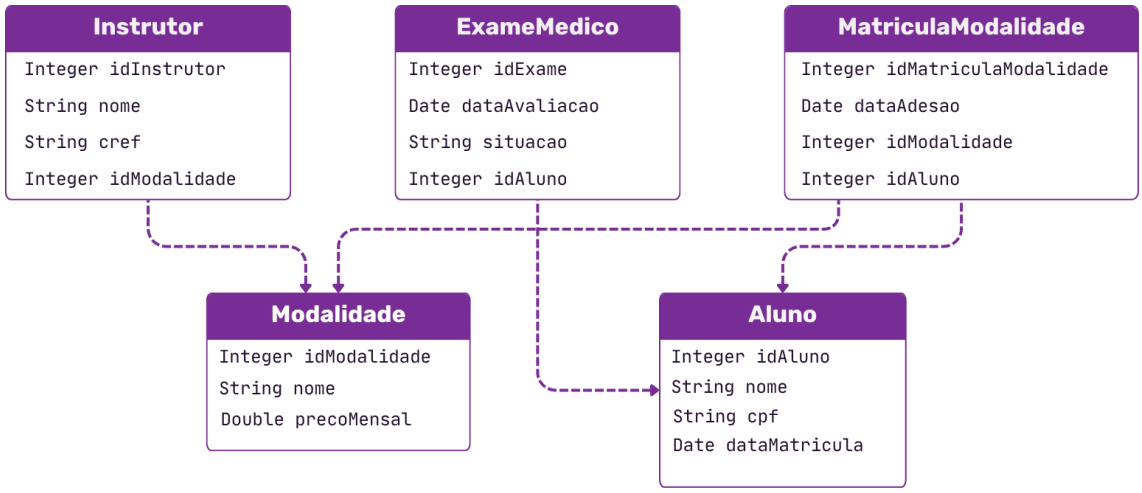


Figura 3: diagrama de classes de domínio

É um diagrama que modela como as entidades do banco de dados são representadas como Classes na linguagem de programação (em Java, C#, Python, etc.) dentro da memória da aplicação.

Classes de domínio: Instrutor, Modalidade, ExameMedico, MatriculaModalidade, Aluno.

Mapeamento de relacionamentos via identificador: Os relacionamentos entre as classes são feitos pelo identificador do objeto relacionado (Integer idModalidade, Integer idAluno), e não por referência direta ao objeto. As setas tracejadas indicam essa dependência: a classe que guarda o id aponta para a classe referenciada.

Cada classe corresponde a uma tabela do modelo relacional (Modalidade → Modalidades, Instrutor → Instrutores, Aluno → Alunos, MatriculaModalidade → MatriculasModalidades e ExameMedico → ExamesMedicos), e cada atributo corresponde a uma coluna, com o nome no padrão da linguagem (por exemplo, precoMensal → preco_mensal).

No código Python, as classes ficam no início do arquivo app.py. Elas só guardam dados: quem as transforma em registros do banco, e vice-versa, é o adaptador (seção 4).

4. Diagrama de Arquitetura de Persistência

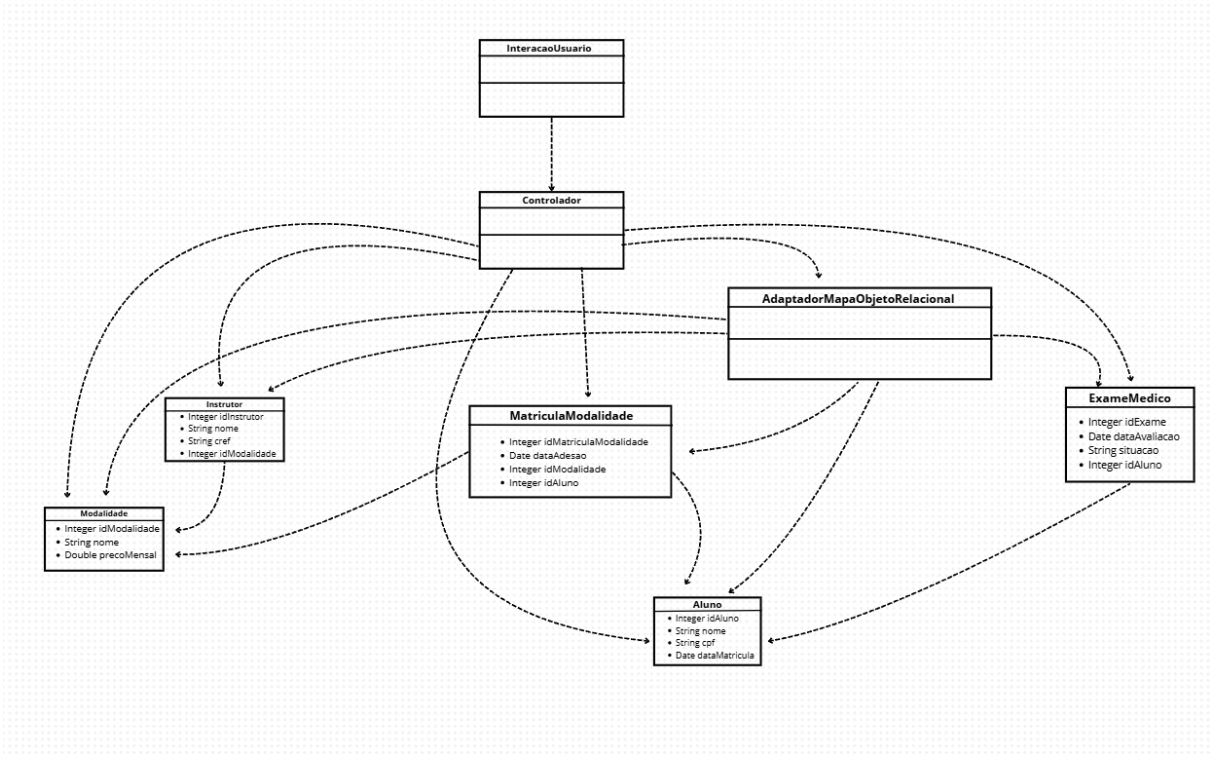


Figura 4: arquitetura de persistência com adaptador objeto-relacional

É o diagrama que mostra como o sistema inteiro se comunica para salvar e recuperar os dados no banco sem bagunçar o código, eliminando a incompatibilidade de impedância (a diferença entre objetos em memória e tabelas no banco).

O fluxo demonstra:

- InteracaoUsuario aciona o Controlador.
- O Controlador manipula os objetos de negócio (Modalidade, Instrutor, MatriculaModalidade, Aluno, ExameMedico).
- O AdaptadorMapaObjetoRelacional é o componente único responsável por serializar/desserializar os objetos e gerar dinamicamente as transações SQL junto ao SGBD.

No código, cada caixa do diagrama é uma classe do arquivo app.py com o mesmo nome. A InteracaoUsuario monta os menus no terminal com a biblioteca questionary, o Controlador cria os objetos de domínio e os repassa ao adaptador, e o AdaptadorMapaObjetoRelacional é a única classe que executa SQL.

O adaptador também decide em qual banco os dados ficam. Ele usa o MySQL como banco principal, tentando conectar em ordem pela biblioteca PyMySQL, pelo mysql-connector e pelo cliente de linha de comando mysql. Se nenhum funcionar, usa um arquivo SQLite

(centro_esportivo.db) como reserva. Ao conectar, cria o banco e as tabelas que ainda não existem e, se o banco estiver vazio, carrega os mesmos 27 registros do script SQL.

5. Implementação do código

Todo o código fonte está no repositório: https://github.com/camilaribeirox/centro-esportivo

O código fica na pasta Sistema Centro Esportivo e tem dois arquivos principais. O centro_esportivo.sql é o script em MySQL 8 que cria o banco e as 5 tabelas, insere os dados de exemplo e executa as consultas, atualizações e remoções. O app.py é a aplicação em Python que implementa a arquitetura da Figura 4. As dependências (questionary e pymysql) estão no requirements.txt, e o README.md explica como executar.

A aplicação abre um menu principal com seis opções. As cinco primeiras correspondem às tabelas (Modalidades, Instrutores, Alunos, Matrículas e Exames Médicos), e cada uma permite listar, cadastrar, atualizar e excluir registros. A sexta opção, Consultas, executa as quatro seleções complexas da seção 6.

5.1 Inserção, seleção, atualização e remoção

Todas as operações seguem o mesmo padrão: o menu lê os dados digitados, o controlador cria o objeto e o adaptador gera o comando SQL. Os valores entram no comando como parâmetros, e não colados no texto, o que evita SQL injection.

- **Inserção (INSERT):** a carga inicial insere 27 registros (5 modalidades, 5 alunos, 5 instrutores, 5 exames e 7 matrículas), sempre começando pelas tabelas que não têm chave estrangeira.
- **Seleção (SELECT):** as listagens leem cada linha da tabela e a transformam em um objeto da classe correspondente.
- **Atualização (UPDATE):** altera um registro pelo seu id. No script SQL também há exemplos de atualização em lote (reajuste de 10% nas mensalidades acima de R$ 100,00) e por CPF.
- **Remoção (DELETE):** apaga um registro pelo seu id. No script SQL há o cancelamento de uma matrícula e a exclusão de um aluno.

5.2 Integridade referencial

As regras das chaves estrangeiras aparecem nas operações de remoção e inserção:

- **ON DELETE CASCADE:** ao excluir um aluno, o banco apaga junto as matrículas e o exame dele, sem deixar registros órfãos.
- **ON DELETE RESTRICT:** uma modalidade que tem instrutor não pode ser excluída. A aplicação mostra uma mensagem de erro em vez de apagar.
- **UNIQUE:** o banco recusa um CPF ou CREF repetido, um segundo exame para o mesmo aluno e a matrícula do mesmo aluno duas vezes na mesma modalidade.

6. Seleções Complexas

As quatro consultas abaixo combinam junção de tabelas (JOIN) com filtros de condição (WHERE e HAVING). Elas estão no script SQL e também na aplicação, no menu 6 - Consultas. Os resultados foram obtidos executando as consultas sobre a carga inicial de dados.

Na aplicação, cada consulta é um método do adaptador, chamado pelo controlador. Na Consulta 1, a mensalidade mínima é digitada pelo usuário. O comando SQL completo de cada consulta está no arquivo centro_esportivo.sql do repositório.

Consulta 1: Ficha Geral de Alunos com Exame Apto (Junção de 5 tabelas)

Realiza o cruzamento de todas as tabelas do centro esportivo para emitir o relatório operacional de alunos ativos e autorizados pelo departamento médico.

- **Junção:** INNER JOIN entre Alunos, MatriculasModalidades, Modalidades, Instrutores e ExamesMedicos. Só aparecem alunos que têm matrícula e exame.
- **Filtros:** WHERE situacao = 'Apto' e preco_mensal >= 90,00.
- **Ordenação:** por nome do aluno e depois por modalidade.

Resultado: 6 linhas: Beatriz (Pilates), Camila (Crossfit), Helena (Crossfit e Musculação) e Lucas (Musculação e Natação), cada uma com o instrutor responsável.

Consulta 2: Auditoria Médica Preventiva (Alunos Inaptos em Atividades Físicas)

Identifica quais alunos possuem matrícula ativa mas estão reprovados no laudo médico, listando a modalidade e o instrutor para bloqueio na catraca.

- **Junção:** INNER JOIN das mesmas 5 tabelas, começando por Alunos e ExamesMedicos.
- **Filtro:** WHERE situacao = 'Inapto', que seleciona só os alunos sem liberação médica.

Resultado: 1 linha: Jeremias Souza, matriculado em Natação com o Prof. Marcelo Costa, está com exame Inapto desde 14/04/2026.

Consulta 3: Faturamento Estimado e Quantidade de Alunos por Modalidade

Mostra o faturamento do centro esportivo e a quantidade de alunos em cada modalidade.

- **Junção:** LEFT JOIN a partir de Modalidades, para que todas as modalidades entrem no agrupamento, mesmo sem aluno.
- **Agregação:** GROUP BY por modalidade, com COUNT (número de alunos) e SUM (receita mensal).
- **Filtro sobre o grupo:** HAVING COUNT(mm.id_aluno) > 0 descarta modalidades sem alunos. Por isso o Boxe não aparece.
- **Ordenação:** da maior para a menor receita.

Resultado: 4 linhas: Crossfit e Natação com 2 alunos e R$ 300,00 cada, Musculação com 2 alunos e R$ 180,00 e Pilates com 1 aluno e R$ 130,00. O Boxe não aparece porque não tem alunos.

Consulta 4: Alunos Multidisciplinares e Mensalidade Acumulada

Localiza alunos que praticam mais de um esporte e calcula o pacote financeiro total de cada um, ou seja, a soma das mensalidades de todas as modalidades em que está matriculado.

- **Junção:** INNER JOIN entre Alunos, MatriculasModalidades e Modalidades.
- **Agregação:** GROUP BY por aluno, com COUNT (quantidade de modalidades), GROUP_CONCAT (nomes das modalidades em uma só coluna) e SUM (mensalidade total).
- **Filtro sobre o grupo:** HAVING COUNT(mm.id_modalidade) > 1 mantém só quem pratica mais de um esporte.

Resultado: 2 linhas: Helena Vieira (Musculação e Crossfit) e Lucas Silva (Musculação e Natação), com R$ 240,00 por mês cada.

7. Conclusão

O trabalho implementou o sistema do centro esportivo desde a modelagem conceitual até o código. O DER e o modelo lógico definem as cinco tabelas e as relações 1:1, 1:N e N:N. O diagrama de classes e a arquitetura de persistência mostram como essas tabelas viram objetos na aplicação, com um adaptador objeto-relacional que concentra todo o SQL. O código implementa seleção, inserção, atualização e remoção, além de quatro seleções complexas com junções de até cinco tabelas e filtros com WHERE e HAVING.